Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — Challenge 3 — Chiffrement et déchiffrement parallèles de texte avec RSA et MPI
4e année Cybersécurité (Ing4-CYB/CCV) - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 22 octobre 2026

Table des matières

Rappel du sujet **2**

1 Partie I : chiffrement et déchiffrement RSA de texte **3**
- 1.1 Organisation du code . . . 3
- 1.2 Fonctions de chiffrement et de déchiffrement . . . 3
- 1.3 Conditions de validité . . . 4
- 1.4 Test séquentiel . . . 4
- 1.5 Variante avec aléa (bonus) . . . 4

2 Partie II : parallélisation avec MPI **5**
- 2.1 Principe . . . 5
- 2.2 Répartition par blocs . . . 5
- 2.3 Choix particuliers . . . 6

3 Partie III : tests et analyse **6**
- 3.1 Tests fonctionnels . . . 6
- 3.2 Protocole de mesure . . . 7
- 3.3 Messages courts : le surcoût de MPI domine . . . 8
- 3.4 Résultats sur de grands textes . . . 8
- 3.5 Analyse . . . 10

Conclusion **11**

Rappel du sujet

Les challenges précédents ont fourni une implémentation des fonctions mathématiques de RSA en C (`rsa.h`, `rsa.c`) et un programme MPI (`rsa_mpi.c`) qui génère en parallèle des paires de clés indépendantes. Ce challenge étend l'application avec le **chiffrement et le déchiffrement parallèles d'un texte** : les caractères du message sont répartis entre les processus MPI, chiffrés indépendamment, collectés dans l'ordre d'origine, puis déchiffrés en parallèle pour reconstituer le message. Les fichiers `rsa.h`, `rsa.c` et `rsa_mpi.c` doivent rester **inchangés**.

Le chiffrement et le déchiffrement d'un caractère reposent sur l'exponentiation modulaire. Pour une clé publique (N, e) et une clé privée (N, d), un caractère de valeur entière m est chiffré puis déchiffré par

$$c = m^e \bmod N, \qquad m = c^d \bmod N.$$

Le sujet demande :

— **Partie I** : une fonction qui chiffre un caractère avec (N, e), une fonction qui déchiffre avec (N, d), la représentation de chaque caractère par sa valeur entière, et la reconversion des valeurs déchiffrées en caractères ;

— **Partie II** : la parallélisation avec MPI : le processus 0 définit le texte et génère une paire de clés avec `generate_rsa_keys()`, les caractères sont distribués, chiffrés indépendamment, collectés au processus 0 dans l'ordre, redistribués pour le déchiffrement parallèle, puis le texte est reconstitué et affiché ;

— **Partie III** : des tests (messages de longueur non divisible par le nombre de processus, plus de processus que de caractères), la vérification de l'identité du texte déchiffré, la mesure des temps pour différents nombres de processus, et une discussion sur la répartition du travail et le surcoût de communication.

1 Partie I : chiffrement et déchiffrement RSA de texte

1.1 Organisation du code

Les fichiers du challenge 2 ne sont pas modifiés. Le code ajouté est organisé ainsi :

— `rsa_text_mpi.c` / `rsa_text_mpi.h` : les fonctions de chiffrement et de déchiffrement d'un caractère (partie I), qui réutilisent `rsa.h` (le fichier d'en-tête inclut `rsa.h`) ;
— `main.c` : le programme MPI (partie II) et les mesures (partie III) ;
— un `Makefile` (cibles `all`, `run`, `run_all`, `bench`, `graphs`), un script `bench.sh` qui lance les mesures et écrit `resultats.csv`, et un script `plot_resultats.py` qui produit les graphes.

1.2 Fonctions de chiffrement et de déchiffrement

L'exponentiation modulaire rapide (*square-and-multiply*) calcule $m^e \bmod N$ en $O(\log e)$ multiplications modulaires :

```
/* Exponentiation modulaire rapide : retourne (base^exp) mod mod. */
long long mod_pow(long long base, long long exp, long long mod) {
    long long resultat = 1;

    base %= mod;

    while (exp > 0) {
        if (exp & 1) {
            resultat = (resultat * base) % mod;
        }
        base = (base * base) % mod;
        exp >>= 1;
    }

    return resultat;
}
```

Chaque caractère est représenté par sa valeur entière. Le caractère est converti en `unsigned char` avant le calcul : le type `char` peut être signé, et un octet supérieur ou égal à 128 (par exemple un caractère accentué en UTF-8) donnerait sinon une valeur négative. La valeur m appartient donc à $[0, 255]$.

```
/* Chiffre un caractere avec la cle publique (N, e) : c = m^e mod N */
long long chiffrer_caractere(char c, long long N, long long e) {
    long long m = (unsigned char)c;

    return mod_pow(m, e, N);
}

/* Dechiffre avec la cle privee (N, d) : m = c^d mod N, puis
   conversion en caractere */
char dechiffrer_caractere(long long c, long long N, long long d) {
```

```
    long long m = mod_pow(c, d, N);

    return (char)(unsigned char)m;
}
```

1.3 Conditions de validité

— **Correction** : comme $e \cdot d \equiv 1 \pmod{\varphi(N)}$ et que $N = pq$ est le produit de deux nombres premiers distincts, on a $m^{ed} \equiv m \pmod{N}$ pour tout $m \in [0, N[$. Le déchiffrement redonne donc exactement m, à condition que $m < N$.

— **Taille de N** : il faut $N > 255$ pour que chaque valeur de caractère soit représentable. Avec la paire $(p, q) = (281, 283)$ utilisée (la dernière du tableau du challenge 2), $N = 79\,523$, $\varphi(N) = 280 \times 282 = 78\,960$, $e = 11$ et $d = 35\,891$ (on vérifie $11 \times 35\,891 = 394\,801 = 5 \times 78\,960 + 1$). Le programme vérifie $N > 255$ avant de commencer.

— **Absence de débordement** : tous les produits intermédiaires sont inférieurs à $N^2 \approx 6{,}3 \times 10^9$, très en dessous de 2^{63}, donc ils tiennent dans un `long long`.

1.4 Test séquentiel

Avec le message `Challenge`, les valeurs chiffrées obtenues sont :

```
Cles : N=79523 e=11 d=35891
Texte original : Challenge
Valeurs chiffrees : 1865 55387 20948 49079 49079 5074 54769 54035
   5074
Texte dechiffre : Challenge
Verification OK
```

Les deux lettres `l` (positions 3 et 4) donnent la même valeur chiffrée 49 079, de même que les deux `e` (5 074). C'est une propriété du RSA « brut » (sans bourrage) : le chiffrement est déterministe, donc deux caractères identiques ont le même chiffré, ce qui rend le texte vulnérable à l'analyse de fréquences.

1.5 Variante avec aléa (bonus)

En pratique, RSA est utilisé avec un bourrage aléatoire (OAEP, PKCS#1) précisément pour éviter ce défaut. Les clés de ce challenge sont trop petites pour OAEP (N fait 17 bits). Nous avons donc ajouté, en complément, une version « jouet » du même principe : on ajoute de l'aléa dans les bits de poids fort du message avant le chiffrement. On construit $m' = 256\,r + m$ avec r aléatoire dans $[0, r_{\max}]$, où $r_{\max} = \lfloor (N - 256)/256 \rfloor = 309$, de sorte que $m' < N$. Le déchiffrement calcule m' puis retrouve le caractère par $m' \bmod 256$.

```
long long chiffrer_caractere_alea(char c, long long N, long long e,
    unsigned int *graine) {
    long long r_max = (N - 256) / 256;
    long long r = rand_r(graine) % (r_max + 1);
    long long m = r * 256 + (unsigned char)c;

    return mod_pow(m, e, N);
```

```
}

char dechiffrer_caractere_alea(long long c, long long N, long long
    d) {
     long long m = mod_pow(c, d, N);

     return (char)(unsigned char)(m % 256);
}
```

Pour `Challenge`, les deux `1` donnent par exemple 40 056 et 4 407 (valeurs différentes, qui changent à chaque exécution) et le texte est correctement retrouvé. Ce n'est **pas** un vrai OAEP : l'aléa est de 8 bits environ et N est factorisable instantanément, donc cette variante n'apporte aucune sécurité réelle. Elle illustre seulement le principe. La version standard, conforme au sujet, reste la version principale ; la variante s'active avec l'argument `alea`.

2 Partie II : parallélisation avec MPI

2.1 Principe

Le programme `main.c` se déroule ainsi :

1. chaque processus initialise MPI et obtient son rang et le nombre de processus (`MPI_Comm_rank`, `MPI_Comm_size`) ;
2. le processus 0 définit le texte (argument de la ligne de commande, ou `Challenge` par défaut) et génère les clés avec la fonction existante `generate_rsa_keys(281, 283, &N, &e, &d)` ;
3. `MPI_Bcast` envoie à tous les processus la longueur du texte, le mode, N, e, d et une graine ;
4. chaque processus calcule la répartition des caractères, puis `MPI_Scatterv` distribue les caractères ;
5. chaque processus chiffre indépendamment ses caractères, puis `MPI_Gatherv` collecte les valeurs chiffrées au processus 0, dans l'ordre d'origine ;
6. `MPI_Scatterv` redistribue les valeurs chiffrées, chaque processus les déchiffre, puis `MPI_Gatherv` collecte les caractères déchiffrés au processus 0 ;
7. le processus 0 reconstitue le texte, le compare à l'original (`memcmp`) et affiche le texte original, les valeurs chiffrées et le texte déchiffré.

2.2 Répartition par blocs

Les n caractères sont répartis en blocs contigus : les $n \bmod P$ premiers processus reçoivent $\lfloor n/P \rfloor + 1$ caractères, les autres $\lfloor n/P \rfloor$. Les tailles de blocs diffèrent donc d'au plus un caractère. Si $P > n$, certains processus reçoivent 0 caractère et restent inactifs sans provoquer d'erreur.

```
static void repartition(int n, int taille, int *comptes, int *
    deplacements) {
     int base = n / taille;
     int reste = n % taille;
```

```
    int position = 0;

    for (int i = 0; i < taille; i++) {
        comptes[i] = base + (i < reste ? 1 : 0);
        deplacements[i] = position;
        position += comptes[i];
    }
}
```

Chaque processus calcule lui-même ces tableaux (le calcul est trivial et évite une communication). On utilise `MPI_Scatterv` et `MPI_Gatherv` plutôt que `MPI_Scatter` et `MPI_Gather`, car ces derniers imposent des blocs de même taille, ce qui est impossible quand la longueur n'est pas divisible par le nombre de processus. L'ordre d'origine est conservé grâce aux déplacements : le bloc du processus r est placé à la position `deplacements[r]`.

```
MPI_Scatterv(c->message, c->comptes, c->deplacements, MPI_CHAR,
    c->local_texte, nb_local, MPI_CHAR, MAITRE, MPI_COMM_WORLD);

for (int i = 0; i < nb_local; i++) {
    c->local_chiffre[i] = chiffrer_caractere(c->local_texte[i], c->
       N, c->e);
}

MPI_Gatherv(c->local_chiffre, nb_local, MPI_LONG_LONG,
    c->chiffre, c->comptes, c->deplacements, MPI_LONG_LONG, MAITRE,
        MPI_COMM_WORLD);
```

2.3 Choix particuliers

— **Graine par processus** (variante avec aléa) : chaque processus utilise la graine de base plus son rang. Sans cela, tous les processus tireraient les mêmes valeurs aléatoires.

— **Types** : les caractères sont transmis en `MPI_CHAR` (1 octet) et les valeurs chiffrées en `MPI_LONG_LONG` (8 octets).

— **Texte généré** : l'argument `@N` génère un texte de N caractères imprimables. C'est nécessaire pour les mesures, car la ligne de commande est limitée à environ 128 Ko par argument.

3 Partie III : tests et analyse

3.1 Tests fonctionnels

Le programme est lancé avec `mpirun -oversubscribe -np` P `./rsa_text_mpi`. Le tableau 1 résume les cas testés. Dans tous les cas, le programme affiche `Verification OK : le texte dechiffre est identique a l'original`.

Message	P	Répartition (caractères par processus)	Résultat
`Challenge` (9 car.)	4	3, 2, 2, 2 (9 non divisible par 4)	OK
`Salut` (5 car.)	2	3, 2	OK
`Salut` (5 car.)	4	2, 1, 1, 1	OK
`Salut` (5 car.)	8	1, 1, 1, 1, 1, 0, 0, 0 (plus de processus que de caractères)	OK
`Bonjour le monde`, variante aléa (16 car.)	8	2 par processus	OK
Texte de 50 000 car.	4	12 500 par processus	OK

TABLE 1 – Tests fonctionnels : longueurs non divisibles et plus de processus que de caractères

À titre d'exemple, voici la sortie pour `Salut` avec 8 processus (trois processus n'ont aucun caractère) :

```
===== RSA texte parallele (8 processus, version standard) =====
Cles : N=79523 e=11 d=35891
Longueur du message : 5 caractere(s)

Repartition :
  processus 0 : 1 caractere(s) [0..0]
  processus 1 : 1 caractere(s) [1..1]
  processus 2 : 1 caractere(s) [2..2]
  processus 3 : 1 caractere(s) [3..3]
  processus 4 : 1 caractere(s) [4..4]
  processus 5 : 0 caractere(s)
  processus 6 : 0 caractere(s)
  processus 7 : 0 caractere(s)

Texte original : Salut
Valeurs chiffrees : 549 20948 49079 45287 5720
Texte dechiffre : Salut

Verification OK : le texte dechiffre est identique a l'original.
```

3.2 Protocole de mesure

Les temps sont mesurés avec `MPI_Wtime()`, après un `MPI_Barrier`, pour chacune des deux phases (chiffrement, déchiffrement) :

- **temps total** : de la distribution (`MPI_Scatterv`) à la collecte (`MPI_Gatherv`) incluses, mesuré par le processus 0 ;
- **temps de calcul** : durée de la boucle de chiffrement (ou de déchiffrement), maximum sur les processus (`MPI_Reduce` avec `MPI_MAX`), car la phase dure aussi longtemps que le processus le plus lent ;
- **temps de communication** : estimé par la différence entre le temps total et le temps de calcul ; il inclut donc aussi les attentes dues à un éventuel déséquilibre.

Pour obtenir des mesures fiables, le programme effectue une **passe à blanc** (qui absorbe le coût d'initialisation de MPI et le remplissage des caches), puis 30 passes mesurées dont on retient la **médiane**. Les tailles testées sont 1 000 000 et 5 000 000 caractères (textes générés), pour $P \in \{1, 2, 4\}$ processus. Comme il n'y a pas de version séquentielle séparée,

T_1 est le temps du même programme lancé avec $P = 1$; l'accélération est $S(P) = T_1/T_P$ et l'efficacité $E(P) = S(P)/P$.

Machine : Dell Inspiron 16, Ubuntu 26.04, OpenMPI, processeur Intel Core i7-11800H (11e génération, 1 socket, 8 cœurs physiques et 16 threads grâce à l'hyperthreading ; fréquence de base 2,3 GHz, jusqu'à 4,6 GHz en mode turbo). `nproc` renvoie 16. Les programmes sont compilés avec `-Wall -Wextra`, sans option d'optimisation.

3.3 Messages courts : le surcoût de MPI domine

Pour un message de 5 caractères, le calcul est négligeable (de l'ordre de 0,001 ms) et la communication domine. Le tableau 2 donne les temps d'une exécution unique, sans passe à blanc :

	Chiffrement (ms)			**Déchiffrement (ms)**		
P	total	calcul	comm.	total	calcul	comm.
2	0,0205	0,0007	0,0197	0,0037	0,0011	0,0026
4	0,0309	0,0005	0,0305	0,0061	0,0006	0,0055
8	0,0365	0,0005	0,0361	0,0094	0,0005	0,0088

TABLE 2 – Texte `Salut` (5 caractères) : le temps total *augmente* avec P alors que le calcul reste constant

Le temps de calcul reste de l'ordre de 0,0005 à 0,001 ms, mais le temps total augmente avec P (de 0,0205 à 0,0365 ms au chiffrement) : la communication collective coûte plus cher quand il y a plus de participants, alors que le travail à répartir est minuscule. Pour de tels messages, la parallélisation est donc *contre-productive*. Le chiffrement est plus lent que le déchiffrement dans ces mesures (par exemple 0,0205 ms contre 0,0037 ms) parce qu'il s'agit du premier appel collectif, qui paie des coûts d'initialisation ; c'est pour cela que les mesures suivantes utilisent une passe à blanc.

3.4 Résultats sur de grands textes

Le tableau 3 résume les résultats pour 1 000 000 et 5 000 000 de caractères.

Taille	P	**Chiffrement (ms)**	**Déchiffrement (ms)**	**Total (ms)**	$S(P)$	$E(P)$
10^6	1	≈ 28	≈ 100	≈ 128	1,00	100 %
10^6	2	$\approx 14{,}5$	≈ 51	≈ 66	1,95	97,7 %
10^6	4	≈ 8	≈ 26	≈ 34	3,74	93,6 %
5×10^6	1	≈ 140	≈ 505	≈ 645	1,00	100 %
5×10^6	2	≈ 75	≈ 257	≈ 332	1,94	97,2 %
5×10^6	4	≈ 43	≈ 134	≈ 177	3,66	91,4 %

TABLE 3 – Temps médians (chiffrement et déchiffrement), accélération et efficacité. Les temps sont des ordres de grandeur lus sur les graphes.

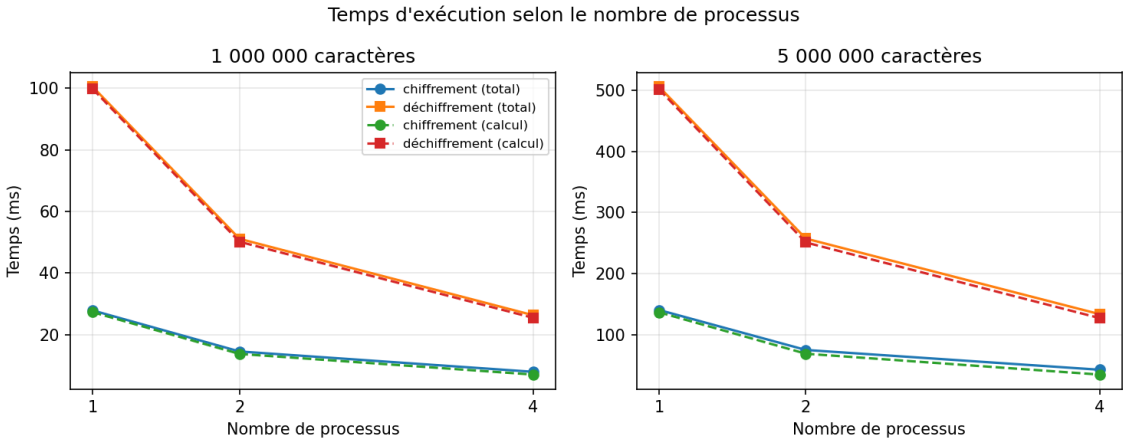


FIGURE 1 – Temps de chiffrement et de déchiffrement (total et calcul seul) en fonction du nombre de processus, pour 10^6 et 5×10^6 caractères

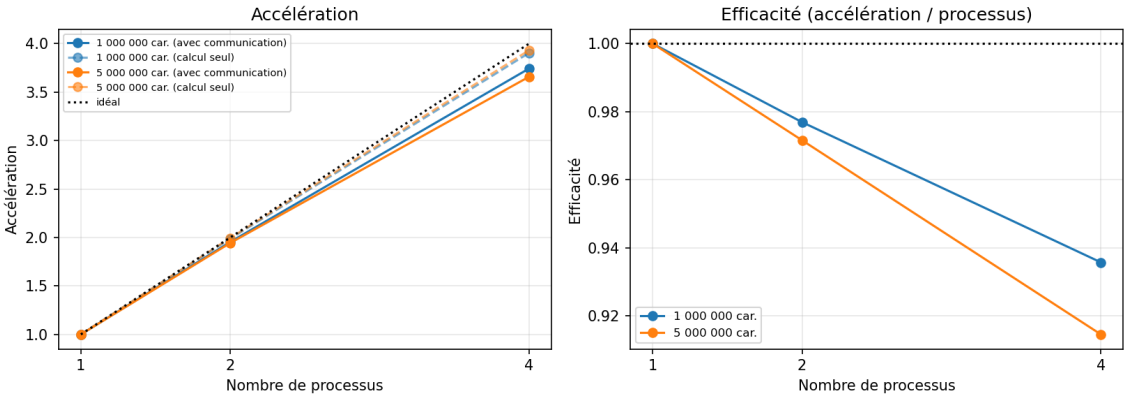


FIGURE 2 – Accélération (avec communication et calcul seul, comparée à l'idéal $S = P$) et efficacité en fonction du nombre de processus

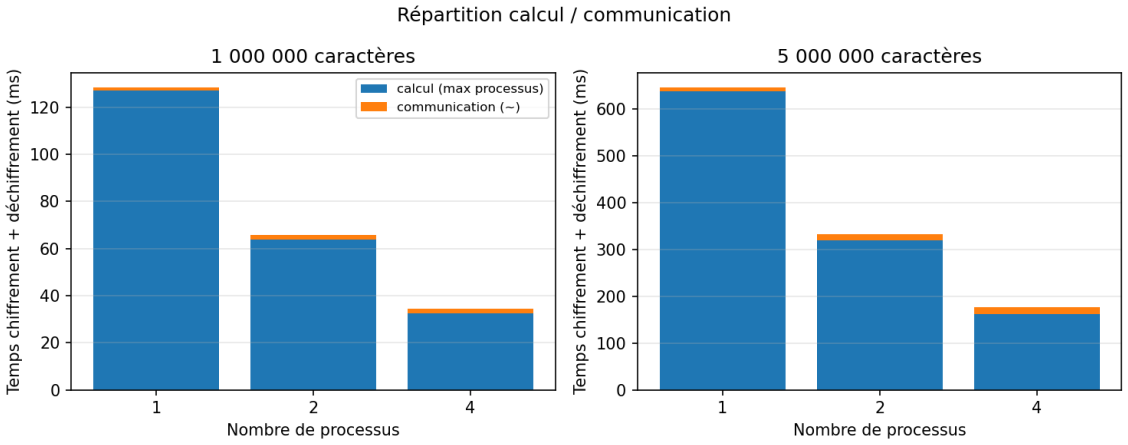


FIGURE 3 – Répartition entre calcul (maximum sur les processus) et communication estimée, chiffrement et déchiffrement cumulés

3.5 Analyse

Passage à l'échelle. Pour de grands textes, le temps diminue presque proportionnellement à P (figure 1). L'accélération atteint environ 1,95 pour $P = 2$ et 3,7 pour $P = 4$ avec 10^6 caractères, et 1,94 et 3,66 avec 5×10^6 caractères. L'efficacité est de 97 % à $P = 2$ et de 91 à 94 % à $P = 4$ (figure 2). Le travail est parfaitement parallèle : chaque caractère est chiffré indépendamment, sans aucune communication pendant le calcul.

Répartition du travail. Contrairement à la génération de clés du challenge 2, la charge est ici *équilibrée par construction* : le coût de $m^e \bmod N$ ne dépend pas de la valeur du caractère, seulement de l'exposant, donc chaque caractère coûte le même nombre d'opérations. Avec la répartition par blocs, les processus ont au plus un caractère d'écart, donc la même charge à moins de $1/n$ près. Le temps de calcul seul (courbes en pointillés de la figure 2) atteint une accélération d'environ 3,9 pour $P = 4$, très proche de l'idéal. Le déséquilibre de charge n'est donc pas la cause de la perte d'efficacité.

Surcoût de communication. La perte d'efficacité vient de la communication, qui reste de l'ordre de quelques pourcents du temps total : environ 1 % pour $P = 1$, 3 % pour $P = 2$ et 6 à 7 % pour $P = 4$ (figure 3), ce qui correspond bien à la perte d'efficacité mesurée (6 à 9 % à $P = 4$). Le schéma de communication est *centralisé* : le processus 0 envoie tous les caractères, reçoit toutes les valeurs chiffrées, puis refait de même pour le déchiffrement. Le volume échangé est proportionnel à n, comme le calcul, et ne diminue pas quand P augmente, alors que le calcul par processus diminue. La part de la communication croît donc avec P. De plus, les valeurs chiffrées sont transmises en `long long` (8 octets), soit 8 fois le volume du texte clair ; comme $N < 2^{32}$, un `unsigned int` (4 octets) divisait ce volume par deux. Cette optimisation n'a pas été testée. Avec 5×10^6 caractères, l'efficacité est légèrement plus faible qu'avec 10^6 ; nous supposons que cela vient de la taille des tampons (plusieurs dizaines de Mo pour les valeurs chiffrées, donc davantage de pression sur la mémoire), mais nous ne l'avons pas vérifié.

Déchiffrement plus lent que le chiffrement. Le déchiffrement dure environ 3,6 fois plus longtemps que le chiffrement (≈ 100 ms contre ≈ 28 ms pour 10^6 caractères sur un processus). Cela s'explique par la taille des exposants : `mod_pow` effectue une itération par bit de l'exposant, avec une multiplication supplémentaire pour chaque bit à 1. Avec $e = 11 = 1011_2$ (4 bits, 3 bits à 1), cela fait $4 + 3 = 7$ multiplications modulaires ; avec $d = 35\,891 = 1000110000110011_2$ (16 bits, 7 bits à 1), $16 + 7 = 23$. Le rapport $23/7 \approx 3{,}3$ est cohérent avec le rapport mesuré. C'est pourquoi, dans RSA, on choisit un petit exposant public.

Initialisation et passe à blanc. Le premier appel collectif de MPI est nettement plus coûteux que les suivants (voir le tableau 2). La passe à blanc avant les mesures sert à ne pas fausser les temps avec ce coût, qui ne dépend pas de la charge. Le temps de `MPI_Init` et de lancement des processus n'est pas inclus dans les mesures.

Limites de l'expérience.

— Les mesures présentées s'arrêtent à $P = 4$; le cas $P = 8$ n'est pas couvert par ces résultats. La machine dispose de 8 cœurs physiques et 16 threads, donc $P \leq 8$ tient sur les cœurs physiques, mais $P = 16$ utiliserait les deux threads de chaque cœur.

— Le placement des processus n'est pas imposé (pas d'option `-bind-to core`) et n'a pas été vérifié.
— La fréquence du processeur (turbo jusqu'à 4,6 GHz) varie avec le nombre de cœurs actifs, ce qui peut modifier légèrement les temps d'une configuration à l'autre.
— Les temps de communication sont estimés par différence : ils incluent les attentes dues à de petits déséquilibres, et le temps total est mesuré par le seul processus 0.
— Les programmes sont compilés sans optimisation, et les résultats dépendent de la taille des textes choisis.

Conclusion

— **Partie I** : `rsa_text_mpi.c` chiffre un caractère par $m^e \bmod N$ et le déchiffre par $c^d \bmod N$, avec une exponentiation modulaire rapide. Les fichiers du challenge 2 sont inchangés. Le RSA brut étant déterministe (deux caractères identiques donnent le même chiffré), nous avons ajouté, en bonus, une variante avec aléa qui illustre le principe du bourrage.
— **Partie II** : le processus 0 génère les clés et les diffuse, les caractères sont distribués par blocs avec `MPI_Scatterv`, chiffrés indépendamment, collectés dans l'ordre avec `MPI_Gatherv`, puis déchiffrés de la même manière. Les longueurs non divisibles par P et les cas $P > n$ sont gérés.
— **Partie III** : tous les tests sont validés (texte déchiffré identique à l'original). Pour de grands textes, l'accélération atteint environ 3,7 avec 4 processus (efficacité de 91 à 94 %). La perte d'efficacité vient de la communication centralisée et non du calcul, dont l'accélération est quasi idéale. Pour des messages courts, le surcoût de MPI domine et la parallélisation est contre-productive.